

---

---

# Session 29 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Github Main Repo Link : [https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks\\_assignments](https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments)

Github File Link : [https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks\\_assignments/blob/main/Session29/Assignment%2029.ipynb](https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session29/Assignment%2029.ipynb)

---

---

---

## Code & Output

---

### 1. Data Preparation

```
[2]: '''  
Q1. Data Preparation  
'''  
  
import pandas as pd  
from sklearn.model_selection import train_test_split  
  
# Load Cleaned Dataset  
df = pd.read_csv("cleaned_emotions.csv")  
  
# Separate Features and Target  
X = df["cleaned_text"]  
y = df["label"]  
  
# Train-Test Split  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.20, random_state=42  
)  
  
# Print Shapes  
print("X_train shape:", X_train.shape)  
print("X_test shape:", X_test.shape)  
print("y_train shape:", y_train.shape)  
print("y_test shape:", y_test.shape)  
  
X_train shape: (12800,)  
X_test shape: (3200,)  
y_train shape: (12800,)  
y_test shape: (3200,)
```

## 2. Bag of Words – Basic

```
[3]: '''
Q2. Bag of Words – Basic
'''

from sklearn.feature_extraction.text import CountVectorizer

# Create CountVectorizer
vectorizer = CountVectorizer()

# Fit and Transform Training Data
X_train_bow = vectorizer.fit_transform(X_train)

# Transform Testing Data
X_test_bow = vectorizer.transform(X_test)

# Print Shapes
print("X_train BoW shape:", X_train_bow.shape)
print("X_test BoW shape:", X_test_bow.shape)

# Display First 20 Feature Names
print("\nFirst 20 Vocabulary Words:")
print(vectorizer.get_feature_names_out()[:20])

X_train BoW shape: (12800, 13359)
X_test BoW shape: (3200, 13359)

First 20 Vocabulary Words:
['aa' 'aaaaand' 'aaaand' 'aac' 'aahhh' 'aaron' 'ab' 'abandon' 'abandoned'
'abandoning' 'abandonment' 'abated' 'abbigail' 'abc' 'abdomen'
'abdominal' 'abducted' 'abhorrent' 'abide' 'abilities']
```

## 3. Bag of Words + MultinomialNB

```
[4]: '''
Q3. Bag of Words + MultinomialNB
'''

from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Create and Train Model
nb_model = MultinomialNB()

nb_model.fit(X_train_bow, y_train)

# Make Predictions
y_pred = nb_model.predict(X_test_bow)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Multinomial Naive Bayes Accuracy:", round(accuracy, 4))

Multinomial Naive Bayes Accuracy: 0.7678
```

## 4. Understanding Vocabulary

```
[5]: '''
Q4. Understanding Vocabulary
'''

# Total Vocabulary Size
print("Total Vocabulary Size:", len(vectorizer.vocabulary_))

# Display Any 15 Words
print("\n15 Vocabulary Words:")
print(vectorizer.get_feature_names_out()[:15])

# Convert One Sample Document into BoW Vector
sample_text = X_train.iloc[0]

sample_vector = vectorizer.transform([sample_text])

print("\nSample Document:")
print(sample_text)

print("\nBoW Vector:")
print(sample_vector.toarray())
```

Total Vocabulary Size: 13359

15 Vocabulary Words:  
['aa' 'aaaaand' 'aaaand' 'aac' 'aahhh' 'aaron' 'ab' 'abandon' 'abandoned'  
'abandoning' 'abandonment' 'abated' 'abbigail' 'abc' 'abdomen']

Sample Document:  
refers course though cant help feeling somehow ironically retrospect loudons son kate mcgarrigle rather talented rufus wainw  
right

BoW Vector:  
[[0 0 0 ... 0 0 0]]

## 5. N-grams with Bag of Words

```
[0]: '''
Q5. N-grams with Bag of Words
'''

# Create CountVectorizer with Unigrams + Bigrams
bigram_vectorizer = CountVectorizer(
    ngram_range=(1, 2)
)

# Fit and Transform Training Data
X_train_bigram = bigram_vectorizer.fit_transform(X_train)

# Transform Testing Data
X_test_bigram = bigram_vectorizer.transform(X_test)

# Print Shape
print("Training Feature Matrix Shape:", X_train_bigram.shape)
print("Testing Feature Matrix Shape:", X_test_bigram.shape)

# Display Some Bigram Features
print("\nSome Bigram Features:")
print(bigram_vectorizer.get_feature_names_out()[:20])
```

Training Feature Matrix Shape: (12800, 91579)

Testing Feature Matrix Shape: (3200, 91579)

Some Bigram Features:

['aa' 'aa full' 'aa meeting' 'aaaaand' 'aaaaand tis' 'aaaand'  
'aaaand stealing' 'aac' 'aac feeling' 'aahhh' 'aahhh work' 'aaron'  
'aaron friends' 'ab' 'abandon' 'abandon project' 'abandon sake'  
'abandoned' 'abandoned ask' 'abandoned believe']

## 6. N-grams + Model Training

```
[7]: '''
      Q6. N-grams + Model Training
      '''

      # Train Multinomial Naive Bayes on Bigram Features
      nb_bigram = MultinomialNB()
      nb_bigram.fit(X_train_bigram, y_train)

      # Predictions
      y_pred_bigram = nb_bigram.predict(X_test_bigram)

      # Accuracy
      bigram_accuracy = accuracy_score(y_test, y_pred_bigram)

      print("Bigram Accuracy:", round(bigram_accuracy, 4))

      # Compare with Q3 Accuracy
      print("Unigram Accuracy:", round(accuracy, 4))

      if bigram_accuracy > accuracy:
          print("Bigram model performed better than the Unigram model.")
      elif bigram_accuracy < accuracy:
          print("Unigram model performed better than the Bigram model.")
      else:
          print("Both models achieved the same accuracy.")

      Bigram Accuracy: 0.7266
      Unigram Accuracy: 0.7678
      Unigram model performed better than the Bigram model.
```

## 7. TF-IDF Vectorization

```
[8]: '''
      Q7. TF-IDF Vectorization
      '''

      from sklearn.feature_extraction.text import TfidfVectorizer

      # Create TF-IDF Vectorizer
      tfidf_vectorizer = TfidfVectorizer()

      # Fit and Transform Training Data
      X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)

      # Transform Testing Data
      X_test_tfidf = tfidf_vectorizer.transform(X_test)

      # Print Shapes
      print("X_train TF-IDF shape:", X_train_tfidf.shape)
      print("X_test TF-IDF shape:", X_test_tfidf.shape)

      # Display First 15 Feature Names
      print("\nFirst 15 TF-IDF Features:")
      print(tfidf_vectorizer.get_feature_names_out()[:15])

      X_train TF-IDF shape: (12800, 13359)
      X_test TF-IDF shape: (3200, 13359)

      First 15 TF-IDF Features:
      ['aa' 'aaaaand' 'aaaand' 'aac' 'aahhh' 'aaron' 'ab' 'abandon' 'abandoned'
       'abandoning' 'abandonment' 'abated' 'abbigail' 'abc' 'abdomen']
```

## 8. TF-IDF + MultinomialNB

```
[9]: '''
      Q8. TF-IDF + MultinomialNB
      '''

      # Create and Train Model
      nb_tfidf = MultinomialNB()
      nb_tfidf.fit(X_train_tfidf, y_train)

      # Make Predictions
      y_pred_tfidf = nb_tfidf.predict(X_test_tfidf)

      # Calculate Accuracy
      tfidf_accuracy = accuracy_score(y_test, y_pred_tfidf)

      print("TF-IDF + MultinomialNB Accuracy:", round(tfidf_accuracy, 4))

      TF-IDF + MultinomialNB Accuracy: 0.6609
```

## 9. Comparison of Vectorizers

```
Q9. Comparison of Vectorizers
'''
comparison = pd.DataFrame({
    "Vectorization": [
        "Bag of Words (Unigrams)",
        "Bag of Words (Unigrams + Bigrams)",
        "TF-IDF"
    ],
    "Accuracy": [
        accuracy,
        bigram_accuracy,
        tfidf_accuracy
    ]
})

print(comparison)

# Find Best Method
best_method = comparison.loc[
    comparison["Accuracy"].idxmax()
]

print("\nBest Method:", best_method["Vectorization"])
print("Best Accuracy:", round(best_method["Accuracy"], 4))

print("\nObservation:")
print(
    f"{best_method['Vectorization']} performed best because it achieved "
    f"the highest accuracy on the test data."
)

\

```

|   | Vectorization                     | Accuracy |
|---|-----------------------------------|----------|
| 0 | Bag of Words (Unigrams)           | 0.767813 |
| 1 | Bag of Words (Unigrams + Bigrams) | 0.726562 |
| 2 | TF-IDF                            | 0.660937 |

```

Best Method: Bag of Words (Unigrams)
Best Accuracy: 0.7678

Observation:
Bag of Words (Unigrams) performed best because it achieved the highest accuracy on the test data.

```



## 10. Mini Project – Complete Vectorization Pipeline

```
[11]: '''
      Q10. Mini Project – Complete Vectorization Pipeline
      '''

      import joblib
      |
      # Load Cleaned Dataset
      df = pd.read_csv("cleaned_emotions.csv")

      # Separate Features and Target
      X = df["cleaned_text"]
      y = df["label"]

      # Train-Test Split
      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.20, random_state=42
      )

      # -----
      # Bag of Words
      # -----

      bow_vectorizer = CountVectorizer()

      X_train_bow = bow_vectorizer.fit_transform(X_train)
      X_test_bow = bow_vectorizer.transform(X_test)

      bow_model = MultinomialNB()
      bow_model.fit(X_train_bow, y_train)
      bow_pred = bow_model.predict(X_test_bow)
      bow_accuracy = accuracy_score(y_test, bow_pred)

      # -----
      # TF-IDF
      # -----

      tfidf_vectorizer = TfidfVectorizer()

      X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
      X_test_tfidf = tfidf_vectorizer.transform(X_test)

      tfidf_model = MultinomialNB()
      tfidf_model.fit(X_train_tfidf, y_train)
      tfidf_pred = tfidf_model.predict(X_test_tfidf)
      tfidf_accuracy = accuracy_score(y_test, tfidf_pred)
```

```

# -----
# Compare Results
# -----

comparison = pd.DataFrame({
    "Vectorizer": ["CountVectorizer", "TfidfVectorizer"],
    "Accuracy": [bow_accuracy, tfidf_accuracy]
})

print(comparison)

# -----
# Select Best Model
# -----

if bow_accuracy >= tfidf_accuracy:
    best_vectorizer = bow_vectorizer
    best_model = bow_model
    best_name = "CountVectorizer"
    best_accuracy = bow_accuracy
else:
    best_vectorizer = tfidf_vectorizer
    best_model = tfidf_model
    best_name = "TfidfVectorizer"
    best_accuracy = tfidf_accuracy

print("\nBest Vectorizer:", best_name)
print("Best Accuracy:", round(best_accuracy, 4))

```

```

# -----
# Save Best Vectorizer and Model
# -----

joblib.dump(best_vectorizer, "best_vectorizer.pkl")
joblib.dump(best_model, "best_model.pkl")

print("\nBest vectorizer saved as: best_vectorizer.pkl")
print("Best model saved as: best_model.pkl")

```

```

      Vectorizer  Accuracy
0  CountVectorizer    0.767813
1  TfidfVectorizer    0.660937

```

```

Best Vectorizer: CountVectorizer
Best Accuracy: 0.7678

```

```

Best vectorizer saved as: best_vectorizer.pkl
Best model saved as: best_model.pkl

```